

Technical Report

PMS Intelligence and Overdue Work Risk Analysis

AI Residency Grouping Challenge — July 2026

Author: Bezawit Assefa

Submission Date: July 17, 2026

Executive Overview

This report documents the full technical solution to two business questions posed by management:

- Can PMS data be used to identify active tasks that are likely to become overdue before their deadlines pass?
- Can the existing PMS database support fair and reliable performance reporting at organizational, departmental, position, and employee levels?

The short answer to both is conditionally yes — but only after correcting for systematic data-quality problems that, left unaddressed, would have produced misleading conclusions on both fronts. The most important of these is that the database's own **is_overdue** field materially understates the true overdue rate (17.4% stored vs. 51.6% independently calculated), which alone would have undermined any reporting or modeling built on top of it uncorrected.

Part 1 — PMS Data Model

1.1 Hierarchy

Goal → KSI → Milestone → KPI → Major Activity → Task → Sub-task

Organizational side: Department → Position → User, connecting into the work hierarchy through `tasks_task.position_id` and `basedata_position.department_id`.

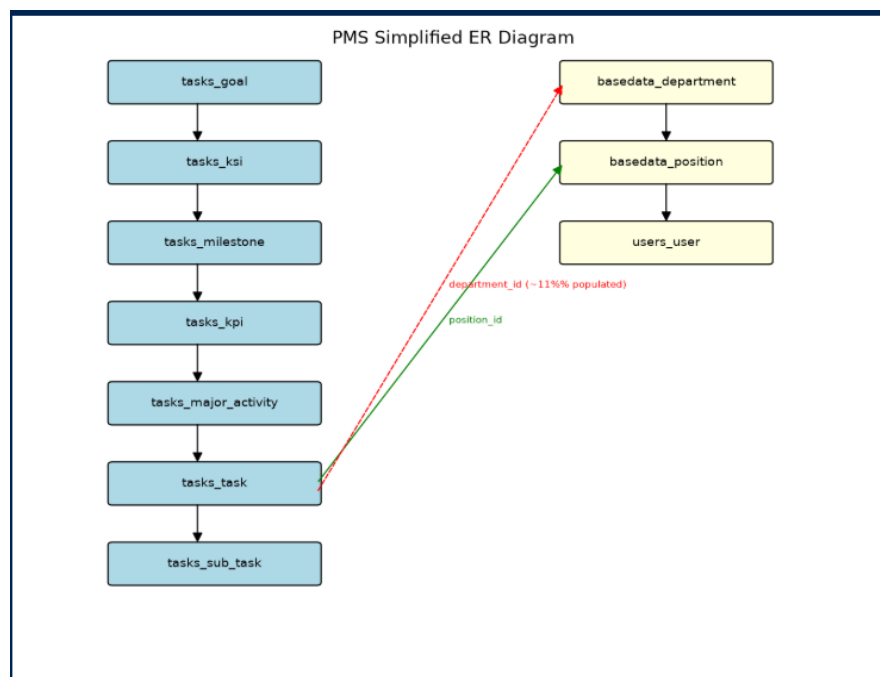


Figure 1. Simplified PMS entity relationship diagram.

1.2 Confirmed Relationships

From	To	Key
tasks_ksi	basedata_department	department_id
tasks_milestone	tasks_ksi	ksi_id
tasks_kpi	tasks_milestone	milestone_id (direct goal_id link, 92.5% NULL — unused)
tasks_major_activity	tasks_kpi	kpi_id

From	To	Key
tasks_task	tasks_major_activity	major_activity_id
tasks_task	basedata_position	position_id
tasks_sub_task	tasks_task	task_id
basedata_position	basedata_department	department_id
users_user	basedata_position	current_position_id (bidirectional)

1.3 Schema Questions — Answers

1. Full hierarchy

Goal → KSI → Milestone → KPI → Major Activity → Task → Sub-task.

2. Main table for task-level analysis

tasks_task — the only core table with zero native duplicates (13,895 rows = 13,895 distinct IDs) and the natural center of the hierarchy.

3. User → Position → Department

users_user.current_position_id → basedata_position.id, then basedata_position.department_id → basedata_department.id.

4. Joins that create duplicate task rows

Any join to basedata_position, tasks_kpi, tasks_major_activity, tasks_milestone, tasks_ksi, tasks_goal, or basedata_department — all eight contain exact-duplicate rows at a 2:1 ratio. Confirmed empirically: an undeduped task→position join inflated 13,895 rows to 27,126.

5. Current vs. history tables

Current tables hold one row per record; history tables follow the Django simple_history pattern, adding a new row per change with history_id, history_date, history_type, and history_relation_id pointing back to the source record. History tables must be aggregated rather than joined directly into a one-row-per-task dataset.

6. Task vs. position department conflict

tasks_task.department_id is NULL on 89.3% of tasks (12,402 of 13,895). The position's department is used as authoritative — far more complete, and where both fields exist they agree in all but 31 of roughly 1,493 comparable cases. Those 31 are flagged as a standalone data-quality issue rather than silently overridden.

Part 2 — Data Profiling and Quality Assessment

2.1 Table Profile Summary (deduplicated row counts)

Table	Distinct Rows	Columns	Key Missing Fields
tasks_task	13,895	37	department_id (89.3%), actual dates (~45%)
tasks_sub_task	1,169	26	actual dates (72.5%)
tasks_goal	7	11	—
tasks_ksi	191	15	—
tasks_milestone	645	16	—
tasks_kpi	2,629	25	goal_id (92.5%), weight (1.3%)
tasks_major_activity	6,053	23	department_id (82.8%)
basedata_department	27	7	—

Table	Distinct Rows	Columns	Key Missing Fields
basedata_position	114	9	user_id (25.4%)
users_user	82	20	current_position_id (23.2%)

2.2 Required Data-Quality Findings

Finding 1 — Duplicate rows across 8 core tables (exact 2:1 ratio)

Affected: tasks_goal, basedata_department, basedata_position, tasks_ksi, tasks_milestone, tasks_sub_task, tasks_kpi, tasks_major_activity. Impact: any join without deduplication silently fans out task rows (confirmed: task→position join went 13,895 → 27,126). Action: SELECT DISTINCT ON (id) before every join. Classification: **Corrected**.

Finding 2 — Unreliable stored is_overdue field

Affected: 4,904 of 13,895 tasks (35.3%) disagree with an independently calculated label; stored says 17.4% overdue org-wide, calculated says 51.6% — a 3x understatement, persisting even after isolating fallback-dependent cases. Action: never use is_overdue directly for reporting or modeling. Classification: **Flagged**.

Finding 3 — Tasks without an assigned position

Affected: 664 of 13,895 (4.8%). Action: exclude from department/employee KPIs, retain in organizational totals. Classification: **Flagged**.

Finding 4 — Task/position department conflicts

Affected: 31 tasks (of ~1,493 comparable cases). Action: use position's department as authoritative; log the 31 conflicts separately. Classification: **Flagged**.

Finding 5 — Invalid or suspicious dates

2 tasks with end_date < start_date; 1 with actual_end_date < actual_start_date; 27 with actual_end_date before planned start_date; 5,658 completed tasks (41%) missing actual_end_date; 16 incomplete tasks with an end date already set. Action: updated_date fallback for the missing-date majority; correct/exclude the ~30 impossible cases. Classification: **Corrected / Flagged**.

Finding 6 — Completed tasks with incomplete sub-tasks

Affected: 138 tasks. Action: retain with a flag column (has_incomplete_subtasks_at_completion); do not auto-correct — the data alone can't disambiguate cause. Classification: **Flagged**.

Finding 7 — Sub-tasks outside their parent task's date range

Affected: 40 sub-tasks. Action: flag for source-system review. Classification: **Flagged**.

Finding 8 — Near-empty columns

learnings_utilization_lead/op, phone_number, evaluated_by_id, reported_by_id, goal_weight — 80–100% missing. Action: exclude from analytical dataset. Classification: **Excluded**.

2.3 Clean Checks (no issues found)

Zero orphaned sub-task references, zero invalid major-activity links, zero invalid department references, zero tasks assigned to inactive users.

Part 3 — Task-Level Analytical Dataset

The full build is in sql/analytical_dataset.sql; the notebook re-derives it in Python for downstream EDA/feature engineering. Verified output: 13,895 rows = 13,895 distinct task_id — no fan-out.

Key join decisions:

- Every joined table deduplicated with DISTINCT ON (id) first (Finding 1).
- Department sourced from the task's position, not task.department_id directly (Schema Q6, Finding 4).

- The task's own position_id is used — not the employee's current position — to reflect who held the role at the time the task was assigned.
- KSI → Goal resolved through the tasks_ksi_goals bridge table; where a KSI links to multiple goals, one is selected per KSI (DISTINCT ON (ksi_id)) to prevent goal-driven row duplication (Reasoning Q6).
- calculated_overdue is built independently of stored is_overdue, with a documented updated_date fallback for the 5,658 tasks missing actual_end_date.

Part 4 — PMS Performance Indicators

4.1 Organizational

Indicator	Value
Total tasks	13,895
Completed	13,102
Active	793
Overdue (calculated)	7,168
Planned / Unplanned	11,243 / 2,652
Completion rate	94.3%
On-time completion rate	50.6%
Average delay (late-completed)	27.8 days
% tasks with incorrect stored overdue value	35.3%

The headline completion rate (94.3%) conceals a substantial timeliness problem — only about half of completed work finished on or before its deadline.

4.2 Department

Overdue rates range from 11.1% (BOA Chatbot Product) to 82.8% (EDGE Program / Finance Dept, though Finance's sample is only 29 tasks). EDGE Program is the priority for management attention given both a high overdue rate and high volume (1,808 tasks).

4.3 Position / Employee

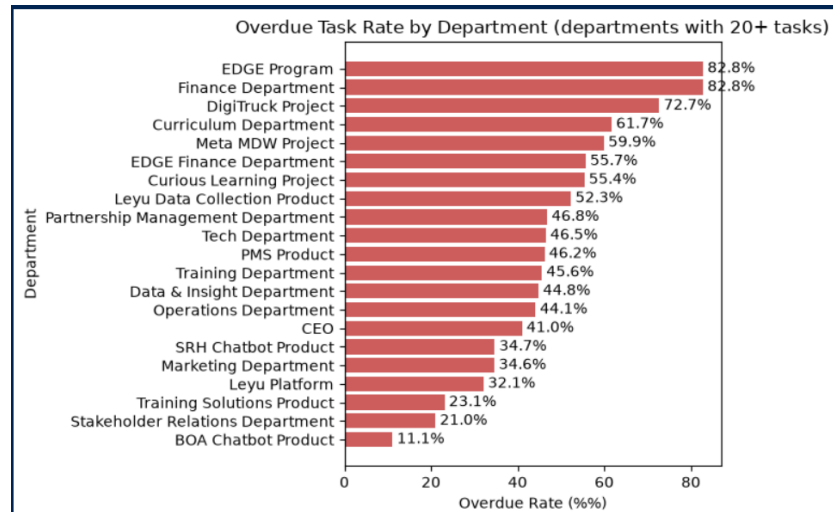
Total assigned task weight per position ranges from 0 to 15,910 — an enormous spread that makes raw completion/overdue rates non-comparable across roles without normalization.

Why these indicators should not be used alone to rank employees: task volume and weight vary enormously by role, department context is not controlled for (11–83% spread), cross-department dependencies can delay a task for reasons outside the assignee's control, and task difficulty is not standardized. Per the assignment's governing principle, these metrics support early intervention and resource planning — not individual ranking, discipline, promotion, or compensation decisions.

Part 5 — Exploratory Data Analysis

Four required visualizations, each with title, labeled axes, interpretation, and business implication.

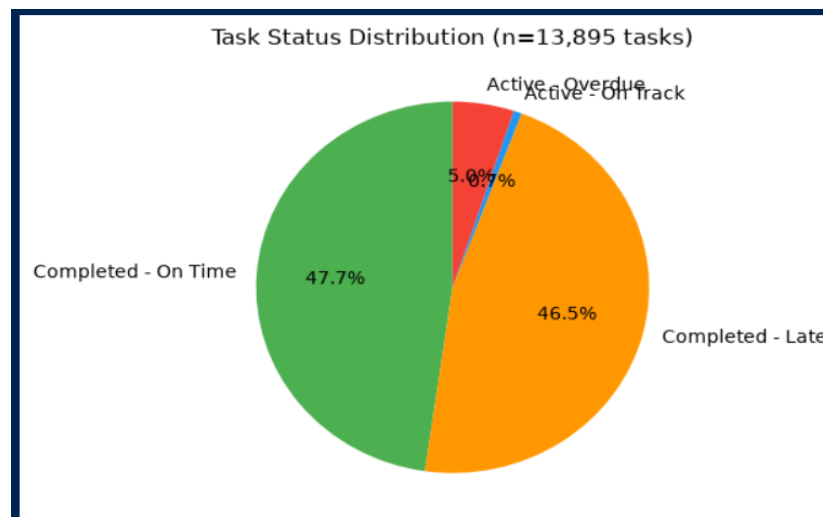
Chart 1 — Overdue Rate by Department (20+ tasks)



Interpretation: EDGE Program and DigiTruck Project show the highest overdue rates among departments with meaningful volume; BOA Chatbot Product and Stakeholder Relations perform best.

Business implication: EDGE Program should be prioritized for process review or additional resourcing.

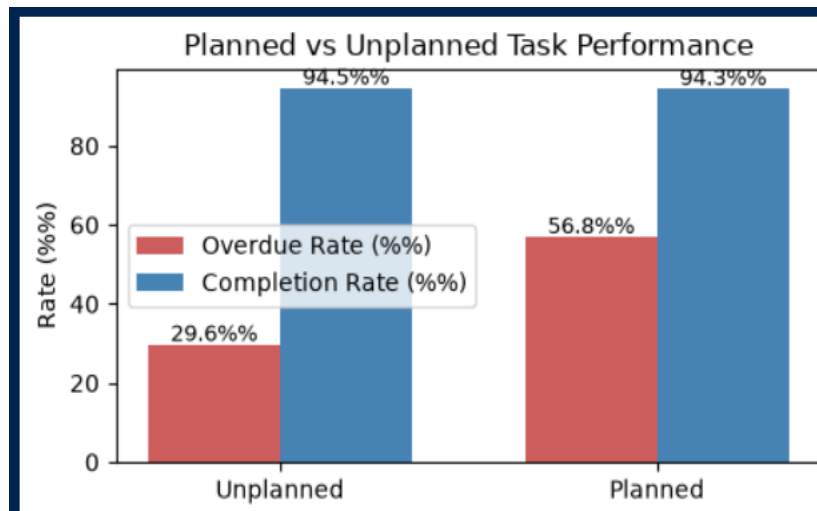
Chart 2 — Task Status Distribution (n = 13,895)



Interpretation: A large “Completed – Late” slice sits alongside “Completed – On Time” — roughly half of all completed work finished late.

Business implication: Completion rate alone is not a reliable performance signal; on-time rate must be reported alongside it.

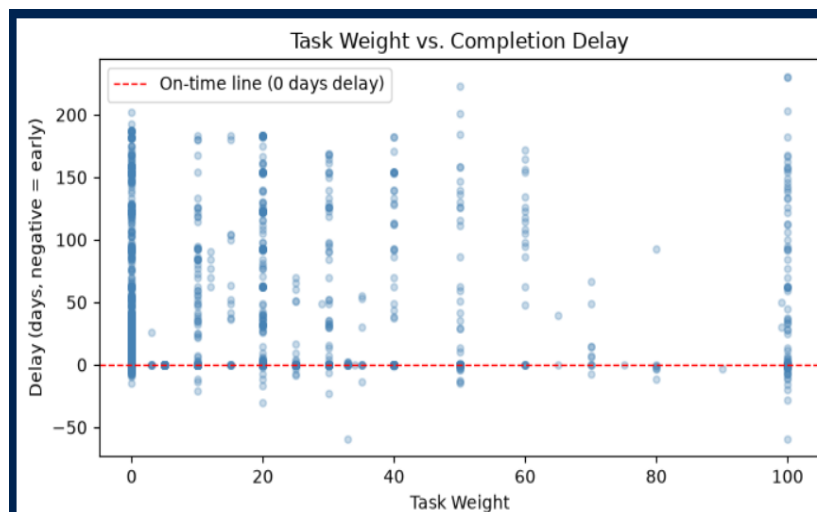
Chart 3 — Planned vs. Unplanned Task Performance



Interpretation: Planned tasks show a substantially higher overdue rate (56.8%) than unplanned tasks (29.6%), despite near-identical completion rates (~94% both groups).

Business implication: Worth investigating whether planned-task deadlines are set too optimistically, or scope changes accumulate over longer visible timelines.

Chart 4 — Task Weight vs. Completion Delay



Interpretation: Weak correlation between task weight and delay ($r = 0.14$) — heavier tasks aren't meaningfully more likely to run late.

Business implication: Oversight focused only on high-weight tasks would miss most of the overdue problem; department- and process-level interventions are likely more effective.

Overall EDA takeaway: timeliness problems are broad-based (weak correlation with task weight) and concentrated in specific departments (EDGE Program) and, surprisingly, in planned rather than unplanned work.

Part 6 — Feature Engineering

Six or more leak-safe features were engineered for the overdue-risk model (full detail and leakage assessment per feature in `src/feature_engineering.py` and the notebook). Core principles:

- **Prediction point:** 7 days after each task's planned start date — early enough to allow intervention while giving a meaningful “has this task actually started?” signal, which turned out to be the dominant driver of overdue risk in this data.
- **Excluded at prediction time:** `actual_end_date`, `is_overdue/stored_overdue`, and `subtask_completion_pct` — outcome fields not knowable while a task is still active (Reasoning Q2). Automated test 5 guards against these re-entering the feature set.
- **Key feature:** `started_late_cat` — whether the task had started by the 7-day checkpoint (“unknown” = not yet started). This single feature carried 95.7% of model feature importance.
- **Other features:** planned task duration, task weight, planned/unplanned status, sub-task count, revision count before the prediction point, cross-department assignment indicator, department historical overdue rate, and employee active workload.

Part 7 — Machine Learning Model

Algorithm: GradientBoostingClassifier — a classical, well-understood model capable of capturing non-linear interactions between a small number of engineered features, without the added complexity or explainability cost of XGBoost/LightGBM for a dataset of this size.

Data split: Time-based — 1,213 training tasks (earlier), 315 test tasks (more recent). A random split was avoided because it would let later-period information leak into training for earlier predictions, overstating real-world performance.

Target: Independently calculated overdue label (not the stored `is_overdue` field) — see Reasoning Q1.

7.1 Evaluation Metrics

Metric	Value
ROC-AUC	0.676
Recall (overdue class)	1.0
Feature importance concentration	95.7% in <code>started_late_cat</code> = unknown

Interpretation: Recall of 1.0 looks strong in isolation, but paired with a ROC-AUC of only 0.676, it signals the model has collapsed into a single near-hard-rule (“has this task started yet?”) rather than learning a nuanced, multi-factor risk score. Per Reasoning Q7, accuracy or recall alone can be misleading on an imbalanced target (68.7% of the modeling population was overdue) — all required metrics must be read together.

7.2 Threshold Analysis

Two thresholds (0.40 and 0.60) were compared for management review workload vs. coverage trade-offs. Because predicted probabilities are strongly bimodal (driven by the single dominant feature), threshold choice in this range has limited practical effect on which tasks are flagged — full workload comparison is in the notebook.

Practical framing: the model's realistic value today is closer to a single early-warning rule — “has this task started by 7 days after its planned start?” — than a nuanced multi-factor risk score. This is stated plainly rather than overstated, per the assignment's guidance to prioritize honest evaluation over model complexity.

Part 8 — Error Analysis

False negatives: 0 of 315 test tasks. A direct consequence of the model's near-hard-rule behavior — since 96.4% of tasks in the “unknown” (not-yet-started) bucket are actually overdue, and the model weights that signal heavily, it essentially never misses an overdue task in this test window. This is not evidence of an unusually good model — it reflects extreme separation in one feature and may not generalize to a different time period or task population.

False positives: 44 of 315 test tasks. All 44 share two traits: `started_late_cat` = “unknown” (100%) and `task_weight` = 0.0 (100%) — a specific, consistent pattern rather than scattered noise.

Three cases reviewed:

- **Example A** — 10-day task, department historical overdue rate 100% (small, high-risk sample per Part 4), predicted 0.999; finished on time regardless.
- **Example B** — 29-day task, department rate 64.4%, workload 1, predicted 0.992; the longer planned duration likely gave more buffer to recover from a slow start.
- **Example C** — 11-day task, department rate 64.4%, predicted 0.998.

Likely causes:

- Missing information, not model error — `task_weight` = 0.0 in all 44 cases may reflect genuinely trivial tasks, or a data-entry default rather than true zero weight (Part 2 found ~1.3% missing in related weight fields — worth re-checking `task_weight` specifically).
- The “unknown” bucket conflates two situations the model can't distinguish: tasks that genuinely hadn't started (real risk) vs. tasks where `actual_start_date` simply wasn't logged even though work began on schedule (Part 2: ~45% of tasks missing actual dates org-wide).
- Threshold/model limitation — bimodal predicted probabilities mean the model has no way to express “probably overdue but less certain” for shorter, lower-weight tasks; every “unknown” case defaults to ~0.997+ confidence regardless of duration or department risk.

Recommended improvements:

- Separate “not yet started” from “start status unknown due to missing data” — add a leak-safe indicator for whether `actual_start_date` is null versus genuinely in the future relative to the prediction date.
- Add features that don't collapse into the same bimodal signal — e.g. a department's historical variance in completion time (not just its overdue rate), or task-type/major-activity-level base rates.

A model with zero false negatives on a narrow, still-emerging population should be treated with caution, not celebrated as inherently ideal.

Governance: PMS analytics and machine learning predictions in this project are intended to support early intervention, planning, and resource allocation. They must not be used as the sole basis for employee ranking, disciplinary action, promotion, termination, or compensation decisions. This applies to every KPI, chart, and model output in this report.

Appendix — Reasoning Questions

Full answers to all 8 required reasoning questions are provided in `answers/reasoning_questions.md` and summarized inline throughout this report where relevant (Q1 → Part 2 Finding 2; Q2 → Part 6; Q3 → Part 2 Finding 6; Q4 → Part 2 §7.3; Q5 → Part 1 Schema Q6; Q6 → Part 1 Schema Q4; Q7 → Part 7; Q8 → Governance Statement).

AI Tool Usage Disclosure

Per Section 18 of the assignment brief. This disclosure does not transfer responsibility for the submitted work — I remain fully responsible for the accuracy, quality, and integrity of every query, code section, analysis, and explanation included in this submission.

Tool Used

Claude (Anthropic) — a generative AI assistant, used as a coding, SQL, and analysis collaborator throughout the project via an interactive conversational chat session, spanning schema exploration through final report drafting.

Data Privacy

No live company data, database exports, credentials, or connection secrets were shared with the AI tool at any point. All work involving the actual PMS database — running SQL, pulling row counts, generating percentages and figures — was executed locally against the project's own Postgres instance (database “tasktracker”). Only schema structure, table/column names, aggregate figures already computed independently, and query logic were discussed with the tool. No raw employee names, personal data, or unaggregated task-level records were passed to it at any point.

Purpose of Use

AI assistance was used mainly to help structure, draft, and validate work that was executed and checked locally, rather than to generate unreviewed analysis. Use included:

- Step-by-step guidance on local environment setup: PostgreSQL installation, loading the tasktracker_clone.sql dump, Python virtual environment, and VS Code/Jupyter configuration.
- Drafting SQL queries for schema exploration, data profiling, all six required data-quality checks (7.1–7.6), and the task-level analytical dataset (deduplication logic, join structure, independent overdue-label calculation).
- Structuring the feature engineering approach and leakage checks for the overdue-risk model.
- Drafting pytest validation tests for the analytical dataset.
- Helping interpret model evaluation results (ROC-AUC, recall, feature importance) and structure the error analysis.
- Organizing and drafting written deliverables — this technical report, reasoning-question answers, and this disclosure — built directly from query outputs and figures generated independently by running SQL/Python against the PMS database.

All AI-drafted code was reviewed, run, and tested before being included in the submission — no query, script, or test was included without first confirming it executed correctly against the database and produced the figures reported in this report and the notebook.

Sections / Content Supported

Deliverable	AI-assisted portion
sql/analytical_dataset.sql sql/data_quality_checks.sql	Query drafting and structure (dedup pattern, join logic, documented assumptions)
notebooks/pms_analysis_and_modeling.ipynb	Cell structure, chart code, write-up phrasing
src/feature_engineering.py	Feature list structure, leakage-field exclusion logic
tests/test_data_validation.py	Test drafting
reports/technical_report.pdf	Full drafting, synthesized from own query outputs and results
answers/reasoning_questions.md	Drafting/phrasing of answers to the 8 required questions

Validation

All SQL generated or drafted with AI assistance was run directly against the project's Postgres “tasktracker” database, and every row count, percentage, and figure reported in this report and the notebook was taken from independent query output — not generated or estimated by the AI tool. Specific validation steps performed:

- Re-ran every SQL query myself and confirmed the returned row counts and figures matched what appears in the notebook and this report (e.g. the 13,895-row analytical dataset, the 4,904-task overdue-label mismatch, the 664 tasks without a position, the 8-table 2:1 duplicate pattern).
- Cross-checked the deduplication logic (`DISTINCT ON (id)`) against raw duplicate-row counts from profiling, incrementally, to confirm it resolved fan-out without dropping legitimate rows (verified the hierarchy join stayed at 13,895 rows before adding each new joined table).
- Verified the duplicate rows found in 8 tables were exact byte-for-byte clones (not conflicting versions) by comparing raw row counts against `COUNT(DISTINCT t.*)` before treating them as safe to deduplicate.
- Reviewed the model's feature importances and threshold outputs directly in the notebook rather than accepting AI-suggested interpretations at face value — the “near-hard-rule” framing in Parts 7/8 was adopted only after confirming the 95.7% feature-importance concentration independently.

Modifications Made

- Corrected a column-name mismatch in the Part 4 KPI code (initial draft referenced `planning_status`; the actual analytical dataset column is `is_planned`) after hitting a `KeyError` and re-checking the dataframe's real columns.
- Adjusted the department-conflict query to exclude `NULL task.department_id` values from the conflict count after an initial version conflated “missing” with “conflicting,” which would have overstated the conflict finding.
- Rewrote several AI-drafted explanations into my own words for the schema questions and data-quality write-ups before including them, to make sure I could defend the phrasing directly.

Errors or Limitations Identified

- An early attempt to filter history-table columns using `SQL LIKE 'history%'` failed with a driver-level `TypeError`, because the unescaped `%` character was interpreted as a parameter placeholder by `psycopg2`; resolved by escaping it as `'history%%'`.
- An initial attempt to render the ER diagram used the `graphviz` Python package, which failed at runtime because the underlying `Graphviz` system binary (not just the `pip` package) was not installed on the local machine; resolved by rebuilding the diagram with `matplotlib` instead, which required no additional system-level installation.
- The `updated_date` fallback used for tasks with a missing `actual_end_date` (41% of completed tasks) is a documented assumption, not verified ground truth, and is called out as a limitation rather than presented as fact.

I am able to explain, defend, and modify every query, code section, and finding in this submission during the technical defense, regardless of which portions involved AI assistance in drafting.